Regression with `statsmodels`

Formula API vs. Matrix API

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 2

Contents

1 Two Interfaces, One Estimator **1**
- 1.1 Setup . . . 1
- 1.2 The Data . . . 1

2 Formula API **2**
- 2.1 Simple Regression . . . 2
- 2.2 Multiple Regression . . . 2
- 2.3 Formula Syntax . . . 2

3 Matrix API **2**
- 3.1 Simple Regression . . . 3
- 3.2 Multiple Regression . . . 3

4 Choosing Between Them **3**

5 Working with the Results Object **4**
- 5.1 Robust Standard Errors . . . 4

1 Two Interfaces, One Estimator

`statsmodels` exposes ordinary least squares through two front ends. The **Formula API** takes an R-style formula string and a `DataFrame`; the **Matrix API** takes a response vector and a design matrix directly. They call the same estimator and return the same results object — the difference is entirely in how the design matrix gets built, and therefore in what can go wrong.

> **Matrix, Formula API: Note the difference.** The Formula API adds the intercept column for you. The Matrix API does **not**. `sm.OLS(y, X)` on a raw `X` silently fits a regression through the origin — no error, no warning, just a different model with a misleading R^2. Wrap the design matrix in `sm.add_constant()`.

1.1 Setup

```
pip install statsmodels pandas
```

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

1.2 The Data

Both examples below use `mtcars`, regressing fuel economy on weight and horsepower.

```
mtcars = sm.datasets.get_rdataset("mtcars", "datasets").data
print(mtcars[['mpg', 'wt', 'hp']].head())
```

```
                   mpg    wt     hp
Mazda RX4         21.0  2.620  110
Mazda RX4 Wag     21.0  2.875  110
Datsun 710        22.8  2.320   93
Hornet 4 Drive    21.4  3.215  110
Hornet Sportabout 18.7  3.440  175
```

`wt` is weight in thousands of pounds, so a coefficient of −5.34 means 1,000 extra pounds costs about 5.3 miles per gallon.

2 Formula API

Formulas are strings: the response on the left of `~`, predictors on the right, joined by `+`. The intercept is implicit.

2.1 Simple Regression

```
model = ols('mpg ~ wt', data=mtcars).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      mpg   R-squared:                       0.753
Model:                              OLS   Adj. R-squared:                  0.745
Method:                   Least Squares   F-statistic:                     91.38
No. Observations:                    32   Prob (F-statistic):           1.29e-10
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     37.2851      1.878     19.858      0.000      33.450      41.120
wt            -5.3445      0.559     -9.559      0.000      -6.486      -4.203
==============================================================================
```

(Output abridged — the full summary also reports AIC/BIC, Durbin–Watson, and the Omnibus/Jarque–Bera normality diagnostics.)

2.2 Multiple Regression

```
model = ols('mpg ~ wt + hp', data=mtcars).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      mpg   R-squared:                       0.827
Model:                              OLS   Adj. R-squared:                  0.815
Method:                   Least Squares   F-statistic:                     69.21
No. Observations:                    32   Prob (F-statistic):           9.11e-12
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     37.2273      1.599     23.285      0.000      33.957      40.497
wt            -3.8778      0.633     -6.129      0.000      -5.172      -2.584
hp            -0.0318      0.009     -3.519      0.001      -0.050      -0.013
==============================================================================
```

Note that the weight coefficient moves from −5.34 to −3.88 once horsepower is included: heavy cars tend to be powerful, so the simple regression was loading part of the horsepower effect onto weight.

2.3 Formula Syntax

The formula string is not merely a naming convenience — it constructs the design matrix, including transformations and categorical expansions, without a separate preprocessing step.

Formula	Meaning
`y ~ x1 + x2`	Additive terms, intercept implicit
`y ~ x1 + x2 - 1`	Suppress the intercept
`y ~ x1 * x2`	`x1 + x2 + x1:x2` — main effects and interaction
`y ~ x1 : x2`	Interaction only
`y ~ np.log(x1)`	Arbitrary Python expressions are evaluated
`y ~ C(cyl)`	Treat as categorical; expand to dummies
`y ~ I(x1 + x2)`	`I()` protects arithmetic from formula syntax

3 Matrix API

The Matrix API takes the design matrix as given. This is more typing and more opportunity for error, but the design matrix is an explicit object you can inspect, cache, and reuse.

3.1 Simple Regression

```
X = mtcars[['wt']]
X = sm.add_constant(X)       # prepend a column of 1s -- REQUIRED
y = mtcars['mpg']

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.753
Model:                            OLS   Adj. R-squared:                  0.745
Method:                 Least Squares   F-statistic:                     91.38
No. Observations:                  32   Prob (F-statistic):           1.29e-10
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         37.2851      1.878     19.858      0.000      33.450      41.120
wt            -5.3445      0.559     -9.559      0.000      -6.486      -4.203
==============================================================================
```

Identical numbers to the Formula API; only the intercept's label changes, from `Intercept` to `const`.

3.2 Multiple Regression

```
X = sm.add_constant(mtcars[['wt', 'hp']])
y = mtcars['mpg']

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.827
Model:                            OLS   Adj. R-squared:                  0.815
Method:                 Least Squares   F-statistic:                     69.21
No. Observations:                  32   Prob (F-statistic):           9.11e-12
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         37.2273      1.599     23.285      0.000      33.957      40.497
wt            -3.8778      0.633     -6.129      0.000      -5.172      -2.584
hp            -0.0318      0.009     -3.519      0.001      -0.050      -0.013
==============================================================================
```

Argument order. `sm.OLS(y, X)` takes the **response first**. This is the reverse of `scikit-learn`'s `fit(X, y)`, and swapping them raises no error when the shapes happen to be compatible — it fits a different regression.

4 Choosing Between Them

Feature	Formula API	Matrix API
Import	`from statsmodels.formula.api import ols`	`import statsmodels.api as sm`
Call	`ols('y ~ x1 + x2', data)`	`sm.OLS(y, X)`
Intercept	Automatic	Manual: `sm.add_constant(X)`
Input	`DataFrame` + formula string	Arrays or `DataFrame`s
Categoricals	`C(col)` expands to dummies	Encode yourself
Transforms	Inline: `np.log(x)`	Apply before the call
Missing data	Dropped rowwise automatically	Your responsibility
Prediction	Pass a raw `DataFrame`	`X_new` must carry the constant
Best for	Exploration, model iteration	Pipelines, generated designs, speed

Rule of thumb. Use the Formula API while you are still deciding what the model is — it makes specification changes one-line edits. Move to the Matrix API when the design matrix is itself computed (rolling windows, factor exposures, lagged panels), when the same design is reused across many fits, or when formula parsing shows up in a profile.

5 Working with the Results Object

Both APIs return the same `RegressionResults`, so everything below is interface-agnostic.

```
print(f"Coefficients:\n{model.params}")
print(f"Std errors:\n{model.bse}")
print(f"t-stats:\n{model.tvalues}")
print(f"p-values:\n{model.pvalues}")
print(f"R-squared:   {model.rsquared:.4f}")
print(f"Adj R-sq:    {model.rsquared_adj:.4f}")
print(f"F-statistic: {model.fvalue:.2f} (p={model.f_pvalue:.2e})")
print(f"Confidence intervals:\n{model.conf_int()}")

fitted    = model.fittedvalues
residuals = model.resid
preds     = model.predict(X_new)    # Matrix API: X_new must include the constant
```

Attribute	Contents
`params`	Estimated coefficients $\hat{\beta}$
`bse`	Standard errors of $\hat{\beta}$
`tvalues`, `pvalues`	t-statistics and two-sided p-values
`conf_int(alpha)`	Confidence intervals; default $\alpha = 0.05$
`rsquared`, `rsquared_adj`	R^2 and its degrees-of-freedom adjustment
`fvalue`, `f_pvalue`	Joint test that all slopes are zero
`fittedvalues`, `resid`	$\hat{y}$ and $y - \hat{y}$
`nobs`, `df_resid`	Sample size and residual degrees of freedom
`cov_params()`	Covariance matrix of $\hat{\beta}$

5.1 Robust Standard Errors

The default standard errors assume homoskedastic, independent errors — an assumption financial data rarely satisfies. Both APIs accept a covariance estimator at fit time, which leaves $\hat{\beta}$ unchanged and recomputes everything downstream of it:

```
model = ols('mpg ~ wt + hp', data=mtcars).fit(cov_type='HC3')       # heteroskedasticity
model = sm.OLS(y, X).fit(cov_type='HAC', cov_kwds={'maxlags': 5})   # + autocorrelation
```

For time-series regressions — returns on factors, say — `HAC` (Newey–West) is the usual choice, and the reported t-statistics will typically shrink. Leaving the default `nonrobust` errors in place is the most common way a published-looking regression overstates its own significance.